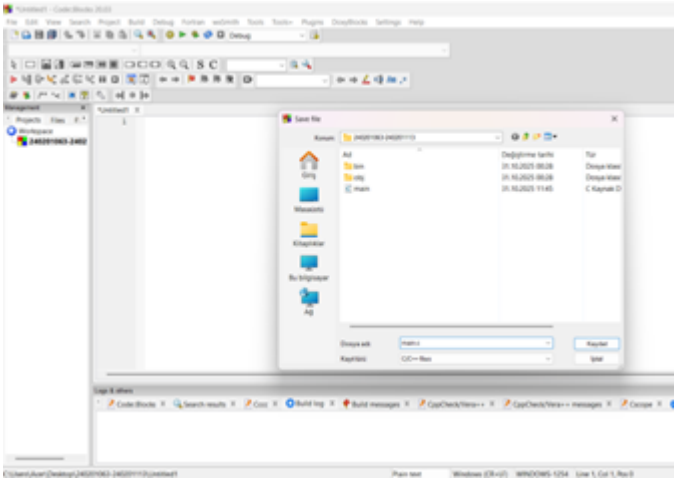
Lidar Verileri ile Hassas Yanaşma ve Geometrik Hesaplamalar: Proje Raporu

Zehra Gülmüş

Ahsen İkbal Türk

***Özet*—Bu çalışmada, Otonom Mobil Robotlarda şarj ünitesine küçük bir hata payı ile yanaşması amaçlanmıştır. Otonom Mobil Robotun gönderdiği ışınlar belli aralıklara göre filtrelenip belirlenen noktalardan doğrular tespit edilmiş, bu doğruların kesişim noktaları bulunup belirli açıdaki kesişim noktaları ile robot arasındaki mesafe hesaplanarak robotun en yakınındaki kesişim noktası bulunmuştur. Filtrelenip işlenen veriler grafik ile görselleştirilmiştir.**



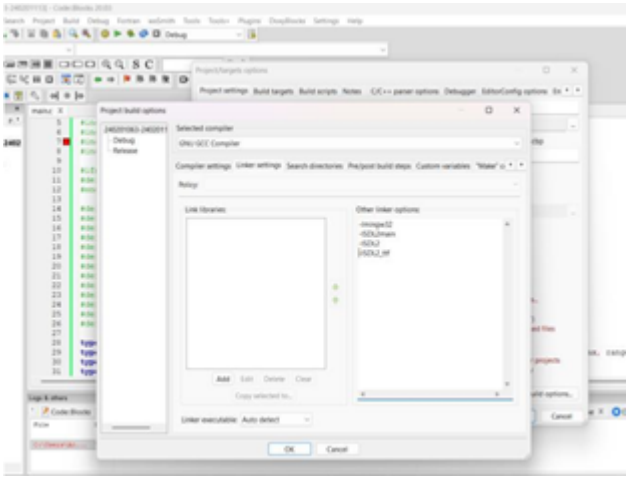
I. Giriş

Otonom mobil robotlar, insanlara ihityaç duymadan kendi başlarına görev yapabilen ve çalışabilen akıllı makinelerdir. Böylece süreklilik gerektiren, tehlikeli veya karmaşık görevleri devralabilirler. Lojistik, üretim, sağlık hizmetleri vb. birçok alanda kullanılırlar. Bu projede amaç, enerji ihtiyaçlarının karşılanması amacıyla robotların şarj istasyonuna hassasiyetle yaklaşabilmelerini sağlamaktır. Bu süreçte LiDAR, kamera ve diğer algılayıcılar kullanılarak robotun çevresini algılayarak hareket etmesi sağlanmaktadır.

Açılımı "Light Detection and Ranging (Işın Algılama ve Mesafe Ölçme)" olan LiDAR, bir başlangıç noktasından başlayarak çevre objelere gönderilen ışınların çarpması ve başlangıç noktasına geri dönmesinde geçen süreden yararlanarak başlangıç noktası ve obje arasındaki mesafenin hesaplanması yöntemine dayanan bir uzaktan algılama teknolojisidir. Fakat verilerin filtreleme, geometrik analiz ve model tahmini ile işlenmesi gerekmektedir.

Bu çalışmada TOML dosyası ile elde edilen LiDAR verileri belli aralıklara göre filtrelenip belirlenen noktalardan doğrular tespit edilmiş, bu doğruların kesişim noktaları bulunup belirli açıdaki kesişim noktaları ile robot arasındaki mesafe hesaplanarak robotun en yakınındaki kesişim noktası bulunmuştur. Filtrelenip işlenen veriler grafik ile görselleştirilmiştir.

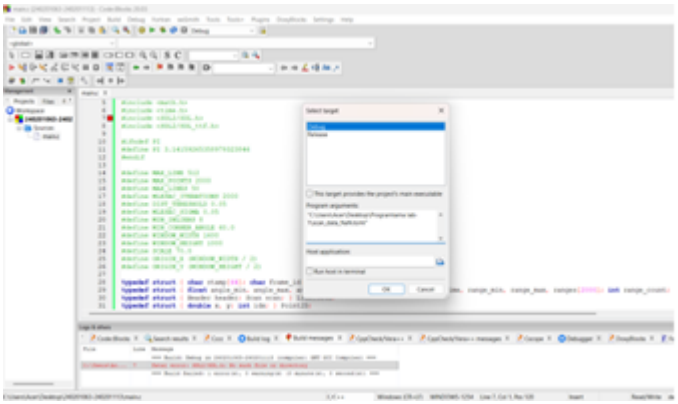
Geliştirilen yazılım C programlama diliyle Code::Blocks programında yazılmış, model tahmini için MLESAC algoritması ve grafiksel gösterim için SDL2 ile SDL_ttf kütüphaneleri kullanılmıştır. İzlenen yöntem ve deneysel sonuçlar raporun devamında daha ayrıntılı bir şekilde ele alınmıştır.



II. Yöntem

A. Dosya Okuma

Programda argüman olarak alınan TOML dosyası açılır ve dosya satır satır okunur. Okuma yaparken boşluk, tırnak, parantez gibi karakterler silinerek başlıklar veya değerler yazı ise string, ondalıklı sayı ise float değişkeni olarak tanımlanır. Köşeli parantezlere gelindiğinde yeni bir bölüme geçileceği anlaşılır. Boş satırlar ve yorum satırları okuma sırasında atlanır. "strtok()" fonksiyonu kullanılarak her bir range değeri belirlenir ve ranges dizisinin eleman sayısı bulunur. Değer belirten ifadeler "key" olarak, değerleri de "value" olarak tutulur. "Lidardata" yapısıyla veriler doğru yere yazdırılır ve dosya kapatılır.



B. Verileri İşleme

TOML dosyasından alınan range yani uzaklık verileri tek tek kontrol edilerek dosyada belirtildiği şekilde uzaklığın alabileceği en küçük ve en büyük değer sınırına uyan ve değeri negatif veya sıfır olmayan mesafeler geçerli range değerleri olarak belirlenir. Geçerli range sayısı bir değişkende tutulur. Daha sonra bu geçerli range uzunluklarının açıları hesaplanır. Değerleri range uzunluğu ve açısından oluşan kutupsal koordinatlardan her bir range değerinin x ve y koordinatları bulunarak Kartezyen koordinat sistemine geçilir.

Kartezyen koordinatlarla ifade edilen noktalardan doğrular oluşturmak için MLESAC (Maximum Likelihood Estimation Sample Consensus) algoritması kullanılmıştır. MLESAC, klasik RANSAC algoritmasının geliştirilmiş bir versiyonu olup, gürültülü verilerde daha güvenilir sonuçlar vermektedir.

İlk aşama olarak rastgele seçilen iki nokta kullanılarak doğru modeli oluşturulur. model oluştururken aynı noktanın iki kez seçilmemesi için bir döngü kullanılır. Genel doğru denklemi şu şekilde ifade edilir: "ax + by + c= 0". Bu denklem ve seçilen iki nokta ile doğruya ait yön vektörü, normal vektör ve doğru sabiti(c) bulunur. Oluşturulan bu doğruya diğer tüm noktaların mesafesi hesaplanır. Eğer mesafe doğruya ait olabilecek uzunlukta ise o nokta inlier, doğruya ait olamayacak kadar uzaksa outlier olarak belirlenir. Bundan sonra inlier noktalar kullanılarak doğru modeli için en küçük kareler yöntemi ile iyileştirme yapılır. Bu aşamada inlier noktaların merkezi hesaplanır, kovaryans matrisi oluşturulur, en büyük özdeğere karşılık gelen özvektör doğrunun en olası yönünü verir. Böylece gürültüye karşı daha sağlam bir doğru modeli elde edilir. Burada MLESAC'ın RANSAC'tan temel farkı noktalar sadece inlier ve outlier olarak ayrılmaz. Bu noktalar da kendi içlerinde kıyaslayarak hangilerinin doğruya daha yakın olduğu belirlenip noktalara doğru modeline en yakın olana en yüksek puan(skor) olacak şekilde bir puanlama yapılır. Hesaplama skoru: $skor = \sum \exp\left(-\frac{mesafe^2}{2\sigma^2}\right)$ şeklinde sürekli bir fonksiyon ile hesaplanır. İki nokta ile doğru oluşturma işlemi 2000 iterasyonluk bir döngüyle tekrarlanarak en iyi uyuma sahip tek bir doğru oluşturulur. Daha sonra bu işlemleri kapsayan diğer bir döngüyle kalan noktalardan diğer doğrular bulunur.

Elde edilen doğrulardan hangilerinin kesiştiğini bulmak için bir fonksiyon kullanılır. Seçilen iki doğrunun ilk ve son noktaları belirlenir. Doğruların eğimleri bulunarak eğer eğimleri aynı ise iki doğru paralel olduğu için kesişmezler sonucuna ulaşılır. Eğer paralel değillerse bu doğruların ya kendileri ya da uzantıları kesişiyor demektir. İlk olarak doğrular sonsuz kabul edilerek iki doğru denkleminin çözümü formülünden bir kesişim noktası belirlenir. Sonra bu kesişim noktasının iki doğrunun da gerçek uzunluğuna dahil olup olmadığı kontrol edilir. Eğer kesişim noktası iki doğrunun da üzerindeyse bu doğrular kesişiyor demektir. Diğer bütün doğruların kesişim durumu main fonksiyonunda döngüyle kontrol edilir.

Kesişen doğruların arasındaki açıyı bulmak için kullanılan fonksiyonda iki doğrunun da yön vektörleri hesaplanır. Bulunan yön vektörleri "açı= arctan(|çarpraz çarpım|/nokta çarpımı)" formülünde kullanılarak doğrular arasındaki açı bulunur. Bu formül aynı zamanda köşelerin her zaman küçük açıyla ifade edilmesini sağlar(Örneğin bir kesişimde 150 derecelik açı değil 30 derecelik açı köşe açısı olarak kabul edilir.). Kodda bu işlem atan2 fonksiyonuna yaptırılarak güvenli hale getirilir. Çünkü eğer açı 90 derece ise hatasız çalışır ve 90 derece sonucu döner.

C. Grafiksel Gösterim

Lidar robotunun konumunu, geçerli noktaları, doğruları, doğruların kesişimlerini ve robota en yakın kesişim noktasını göstermek için C dilinde grafik kütüphanesi olarak SDL2 kullanılmıştır. Öncelikle gerçek dünya koordinatları grafik penceresinde çizim yapmak için koordinatlarına çevrilir. "SCALE" sabiti kullanılarak metreler piksel cinsine dönüştürülür.

Grafik ekranı beyaz arka planla kaplanır. "SCALE" sabiti kullanılarak her 1 metrede bir, çok açık renkte yatay ve dikey x ve y eksenleri olarak ızgara çizgileri çizilir. Robotun orijininden geçen çizgiler ana eksenler olarak daha koyu renkte çizilir. Her bir ızgara çizgisine x ve y eksenleri boyunca orijine olan uzaklıkları sayı olarak yazılır. Bunun için "TrueTypeFont(TTF)" kullanılır. Böylece robotun (0,0) konumunda bulunduğu (x,y) koordinat sistemi hazırlanmış olur.

Geçerli Lidar noktaları çizim modu "blending(karıştırma)" moduna alınarak 4x4 boyutunda yarı saydam kareler şeklinde gösterilir. Kümelenme nedeniyle üst üste binen noktalar daha koyu ve belirgin görünür. Robot opak kırmızıyla belirgin bir daire çizdirilerek gösterilir.

Doğruların kesiştikleri yerlerde çakışık görünmelerini önlemek ve köşelerini belirtmek için belirlenen kısaltma miktarı kadar içe doğru çekilir. Bunun için doğruların yön vektörleri uzunluklarına bölünerek yön vektörleri bulunur. Kısaltma işlemi doğrunun başlangıç noktasından birim vektör yönünde, bitiş noktasından da birim vektörün tersi yönünde yapılır.

Doğrular 10 farklı SDL_Color yapısından oluşan bir renk paleti tanımlanarak her biri farklı renkte olacak şekilde çizdirilir. Doğrular opak renkte çizdirilir. Noktalardan MLESAC algoritmasıyla seçilen doğrulara ait noktalar da 6x6 piksel boyutunda kendi doğrularıyla aynı renkte opak olarak gösterilir.

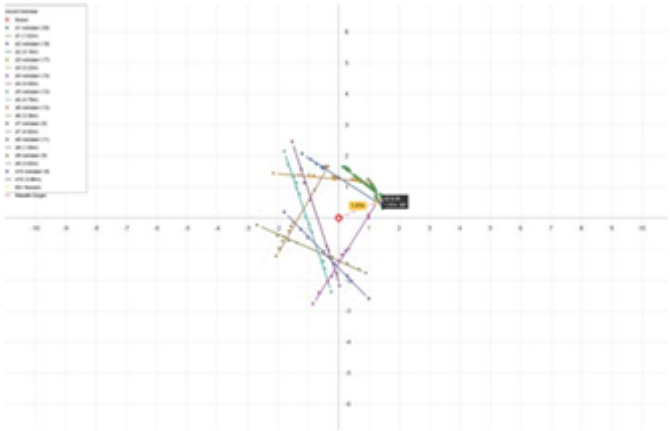
Bütün geçerli kesişimlerin robota olan mesafeleri hesaplanır, mesafe bilgileri tutulur ama robota en yakın mesafe

kesikli çizgiyle gösterilir. Robota en yakın mesafedeki kesişim noktasının robota uzaklığını göstermek için; robot ve kesişim noktasının koordinarlarından mesafe hesaplanır. Mesafe uzunluğunun birim vektörü bulunur. Mesafeyi kesikli çizgiyle göstermek için birim vektör yönünde belirli uzunlukta kısa bir çizgi çizilir ve belirli uzunlukta bir boşluk atlanır. Mesafe boyunca bu işlem tekrar ettirilir.

Geçerli kesişim köşelerini göstermek için her bir köşeye siyah bir bilgi kutusu etiketi yapıştırılır. Bilgi kutusunun genişliği iki metin satırından en uzun olanına göre, yüksekliği ise iki metnin yüksekliği ve aralarındaki boşluğa göre otomatik olarak ayarlanır. TTF kütüphanesiyle bilgi kutularına yazılacak hangi doğruların kesiştiği, aralarındaki açı ve robota olan mesafe bilgileri metin olarak oluşturulur sonra çizime hazır "SDL_Surface" nesnelerine dönüştürülür. En son metin yüzeyleri dokulara çevrilir ve siyah bilgi kutularına yapıştırılır.

Robota en yakın köşeyi gösteren kesikli çizginin orta noktası, mesafenin ilk ve son noktasının koordinatları toplanıp ikiye bölünerek bulunur. Bu orta noktanın 10 piksel yukarısına sarımsı-turuncu bir etiket konur ve üstüne TTF ile oluşturulan mesafe bilgisi yapıştırılır.

Grafiğin sol üst köşesine bilgilendirici bir tablo(Legend) çizdirilir. Bu tablo grafikteki her sembolün, çizginin ve rengin ne anlama geldiğini açıklar. Tablo yarı saydam beyaz bir kutu olarak çizilir ve etrafına gri bir çerçeve eklenir. Tablonun yüksekliği MLESAC ile bulunan doğruların sayısına göre dinamik olarak hesaplanır. Ayriyeten doğrulara ait noktaların sayısı da tabloda belirtilir.



III. Deneysel Sonuçlar

Geliştirilen yazılım, projenin açıklamasında örnek olarak verilen scan_data_NaN.toml dosyasında test edilmiştir. Programda minimum ve maksimum değerlerin dışında olan veriler filtrelenmiştir.

Veriler filtrelendikten sonra noktaların kutupsal koordinatları kartezyen koordinatlara dönüştürülmüştür. MLESAC algoritması kullanılarak benzer doğrular üzerinde olan noktalar belirlenmiştir. Doğrular en az 8 nokta içermektedir.

Tespit edilen doğrulardan kesişenler belirlenip bu kesişimlerin açıları hesaplanmıştır. 60° ve üzeri açılara sahip olan kesişim noktalarından en yakın olan robotun hedef konumu olarak seçilmiştir. Robotun konumu (0,0)'dır.

Kullandığımız algoritma dolayısıyla örnek dosya her çalıştırıldığında farklı sonuçlar elde edilmiştir. Her seferinde 9-11 adet doğru tespit edilmiştir. Doğruların kesişim noktaları robota ortalama 1-6 m mesafede çıkmıştır. Bu da verilerin fiziksel ortamla uyumlu olduğunu göstermektedir.

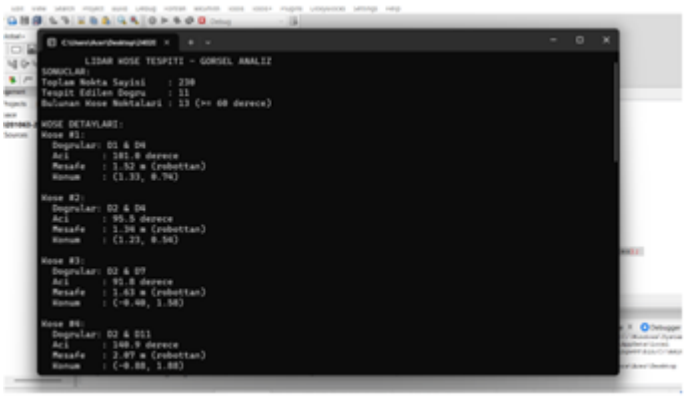
SDL2 ve SDL2_ttf kütüphaneleri kullanılan grafiksel gösterimde robot, doğrular, doğrularda kullanılan ve kullanılmayan noktalar aynı koordinat sistemi üzerinde çizdirilmiş; kesişim koordinatları, açı değerleri ve mesafe bilgileri görsel olarak ekrana yazdırılmıştır. Elde edilen sonuçlar algoritmanın doğrusal bölgeleri tespit etmeyi başardığını göstermiştir. Grafiksel çıktılar, robotun çevresindeki fiziksel alanın somutlaştırılmasına olanak sağlamıştır.

IV. Sonuçlar

Bu çalışmada hem günlük hayatta otonom mobil robotlarda uygulanabilecek bir program ortaya konulmuş hem de geliştirilmesinde rol alanlar için çok öğretici bir deneyim olmuştur. Robotlardaki tek bir özellik için ne kadar emek sarf edildiğini biz geliştiricilere göstermiştir.

Bu projede yer almak bize LiDAR sisteminin kullanım alanlarının genişliğini, verilerin işlenmesi için algoritmaların kullanıldığını ve kullanılan bu algoritmaların çeşitliliğini öğretmiş, grafik kullanımı için nasıl bir yol izleyebileceğimizi göstermiştir.

Özetle bu proje, teorik bilgilerden dolayı soyut kalan programlamanın günlük hayatta nasıl kullanıldığına dair doğrudan fikir edindirmiştir.



V. Algoritmanın Kaba Kodu

Aşağıda, projede kullanılan temel algoritmaların işleyiş mantığı kaba kod (pseudocode) formatında sunulmuştur.

```
PROGRAM LidarKoseTespiti
SABITLER: MAX_POINTS = 2000, MAX_LINES = 50
MLESAC_ITERATIONS = 2000, DIST_THRESHOLD =
0.05 MIN_INLIERS = 8, MIN_CORNER_ANGLE = 60.0

VERI YAPILARI: Point2D: {x, y, indeks}
    LineModel: {a,
```

```
b, c, inlier_sayısı, inlier_indeksleri[]} Line
    :
{başlangıç_noktası, bitiş_noktası, model}
ValidIntersection: {kesişim_noktası, açı,
    mesafe, doğruA,
doğruB}

FONKSİYON LidarVerisiYukle(dosya): Dosyayı aç
    ve
TOML formatında oku header, scan
    parametrelerini ve
ranges dizisini çıkar DÖN lidar_verisi

FONKSİYON MenzilleriFiltreleVeDonustur(
    lidar_verisi):
geçerli_noktalar = [] HER menzil İÇİN ranges:
    EĞER
menzil geçerli ve sınırlar içindeyse: açı =
    angle_min +
indeks × angle_increment x = menzil × cos(açı)
     y = menzil
× sin(açı) geçerli_noktalar'a ekle DÖN geç
    erli_noktalar

FONKSİYON NoktaDogruMesafesi(model, nokta): DÖ
    N
|a×x + b×y + c|

FONKSİYON MLESACEnIyiDogruBul(noktalar[]):
en_iyi_skor = -∞ TEKRAR ET MLESAC_ITERATIONS
kez: Rastgele 2 nokta seç model = İki noktadan
     doğru
denklemi oluştur (ax + by + c = 0)

skor = 0
inliers = []
HER nokta İÇİN:
mesafe = NoktaDogruMesafesi(model, nokta)
```

$$skor += \exp\left(-\frac{mesafe^2}{2\sigma^2}\right)$$

```
EĞER mesafe < DIST_THRESHOLD İSE:
inliers'a ekle

EĞER skor > en_iyi_skor İSE:
en_iyi = {model, skor, inliers}
EĞER en_iyi.inliers >= MIN_INLIERS İSE:
    model = EnKüçükKarelerleİyileştir(en_iyi.
    inliers)
    DÖN (başarılı, model, inliers)
DÖN başarısız

FONKSİYON TumDogrulariCikart(noktalar[]): doğ
    rular
= [] kalan_noktalar = noktalar

DÖNGÜ (doğru_sayısı < MAX_LINES VE yeterli
    nokta
varsa):
    (başarı, model, inliers) =
MLESACEnIyiDogruBul(kalan_noktalar)
    EĞER başarısızsa: ÇIK

    En uzak iki inlier'ı bul → başlangıç ve
    bitiş noktası
    doğru = {başlangıç, bitiş, model, inliers}
    doğrular'a ekle
    İnlier'ları kalan_noktalar'dan çıkar

DÖN doğrular

FONKSİYON DogruKesisimiVarMi(doğru_a, doğru_b)
    :
Parametrik denklemlerle kesişim noktasını
    hesapla EĞER
kesişim her iki doğru segment içindeyse: DÖN (
    evet,
kesişim_noktası) DÖN (hayır, null)

FONKSİYON DogrularArasiAci(doğru_a, doğru_b):
    Yön
vektörlerini kullanarak açıyı hesapla DÖN açı
    (derece)

FONKSİYON KoseNoktalariBul(doğrular[]): köş
    eler = []
HER doğru_i İÇİN doğrular: HER doğru_j İÇİN (i
    +1'den
başlayarak): (kesişiyor, nokta) =
    DogruKesisimiVarMi(i,
j) EĞER kesişiyorsa: açı = DogrularArasiAci(i,
     j) EĞER
açı >= MIN_CORNER_ANGLE İSE: mesafe =
sqrt(nokta.x^2 + nokta.y^2) köşe = {nokta, açı,
    mesafe, i, j}
köşeler'e ekle DÖN köşeler

FONKSİYON DunyadanEkranaDonus(x, y): ekran_x =
ORIGIN_X + (x × SCALE) ekran_y = ORIGIN_Y - (y
     ×
SCALE) DÖN (ekran_x, ekran_y)

FONKSİYON GorselCiz(renderer): IzgaraCiz() //
    Beyaz
arka plan, gri ızgara NoktalariCiz() // Açık
    mavi noktalar
DogrulariCiz() // Renkli kalın çizgiler
DogruNoktalariniCiz() // Renkli inlier
    noktalar
RobottanKoselerezgiCiz() // Kırmızı kesikli ç
    izgiler
MesafeEtiketleriCiz() // Sarı etiketler
    KoseBilgileriCiz() //
Sarı X + bilgi kutusu RobotuCiz() // Kırmızı
    daire
LegendCiz() // Sol üst açıklama kutusu

BAŞLA: lidar_verisi = LidarVerisiYukle(
    dosya_adı)
noktalar = MenzilleriFiltreleVeDonustur(
    lidar_verisi)

doğrular = TumDogrulariCikart(noktalar)
köşeler = KoseNoktalariBul(doğrular)

KONSOLA YAZDIR:
    - Toplam nokta sayısı: {nokta_sayısı}
    - Tespit edilen doğru: {doğru_sayısı}
    - Bulunan köşe: {köşe_sayısı}
    - Her köşe için: doğru çifti, açı, mesafe,
     konum

SDL penceresi oluştur
```

```
en_yakin_kose = Robota en yakın köşeyi seç

DÖNGÜ (program çalışırken):
    Olayları kontrol et (ESC, kapatma)
    GorselCiz(renderer)
    Ekranı güncelle

Kaynakları temizle

BİTİR
```

KAYNAKLAR

[1] Locus Robotics, "What Are Autonomous Robots? 8 Application For Today's AMRs," [Online]. Erişim: What are Autonomous Mobile Robots: AMR Examples & Uses

[2] başarsoft, "Lidar Nedir?," [Online]. Erişim: LiDAR - LiDAR Nedir? LiDAR Sensörü ve Drone | Başarsoft

[3] codeincodeblock, "Setup SDL2 in Code::Blocks IDE," [Online]. Erişim: Setup SDL2 in Code::Blocks IDE

[4] Index of /release, [Online]. Erişim: Index of /release

[5] libsdl-org/SDL_ttf, [Online]. Erişim: Release 2.20.2 · libsdl-org/SDL_ttf · GitHub